

How the Wolf and Lamb tables were built

27 September 2026

Abstract

This note gives the exact algorithms behind the solution tables of Wolf and Lamb: how a position is turned into a table index, how the game is cut into layers by lamb count, the two breadth-first retrograde passes that assign every position its value and exit distance under the 100-turn rule, the second solver that adds the number of plies to the end of the game, the 2-bit packing used by the website, and the independent verifier. Everything here is what the programs `solver/solve.c`, `solver/dist.c`, `solver/pack.js` and `solver/verify.js` actually do.

1 States and the rules that matter

Cells are numbered $0, \dots, 24$ with $\text{cell} = 5(r-1) + (c-1)$ for board coordinates (r, c) , $r, c \in \{1, \dots, 5\}$. A *state* is $s = (w, \ell, \mu)$: w the set of the three wolf cells, ℓ the set of lamb cells, $\mu \in \{\text{W}, \text{L}\}$ the side to move. Sets are 25-bit masks. The no-capture counter and the repetition history are deliberately *not* part of the state; Section 4 explains why the tables are nevertheless exact under both rules.

Rules used by the solver: a step moves a piece to an orthogonally adjacent empty cell; a wolf may instead jump over one empty cell in a row or column onto a lamb, which is removed; a side with no legal move loses; the wolves win when three or fewer lambs remain; 100 plies without a capture is a draw; threefold repetition is a draw.

2 Indexing: from a state to a table entry

Symmetry. The eight symmetries of the square (the dihedral group D_4) preserve every rule, so two states that differ by a symmetry have the same value. Each symmetry is stored as a permutation of the 25 cells; to apply one to a mask quickly, five lookup tables of 32 entries map each 5-bit row of the mask to its image, and the images are OR-ed.

Wolf orbits. The $\binom{25}{3} = 2300$ wolf triples are enumerated and ranked by the combinatorial number system,

$$\text{rank}_3(\{a < b < c\}) = \binom{a}{1} + \binom{b}{2} + \binom{c}{3} \in \{0, \dots, 2299\}.$$

For each triple w its *canonical form* $\text{canon}(w) = \min_{g \in D_4} g(w)$ (numerically smallest mask) is computed; distinct canonical forms are numbered $o = 0, \dots, 318$ in order of first appearance. For every raw triple the solver stores its orbit number $\text{orb}(w)$ and the list of symmetries g with $g(w) = \text{canon}(w)$ (its *coset*; usually one element, up to eight when the triple is itself symmetric).

Lamb rank. Fix an orbit o with canonical wolf mask W_o . The 22 non-wolf cells are numbered $0, \dots, 21$ in increasing cell order; a k -subset $\{c_0 < c_1 < \dots < c_{k-1}\}$ of them has colex rank

$$\text{rank}(\ell) = \sum_{i=0}^{k-1} \binom{c_i}{i+1} \in \left\{0, \dots, \binom{22}{k} - 1\right\},$$

which is a bijection onto that range and is monotone in the numeric value of the compressed mask. Ranking costs one binomial lookup per lamb; unranking runs i from $k - 1$ down to 0 and takes the largest c with $\binom{c}{i+1} \leq$ the remaining rank.

Index. For a state (w, ℓ, μ) with $k = |\ell|$:

1. $o = \text{orb}(w)$; among the coset symmetries g choose the one minimising $g(\ell)$ and put $\ell' = g(\ell)$ (so that a symmetric wolf triple still yields a unique representative);
2. $\text{idx} = (o \cdot \binom{22}{k} + \text{rank}(\ell')) \cdot 2 + \lfloor \mu = L \rfloor$.

A layer with k lambs therefore has $N_k = 319 \cdot \binom{22}{k} \cdot 2$ entries. Two states get the same index if and only if they are symmetric images of each other. Decoding an index back to a state (needed by the solver when it processes a queue of indices) inverts the formula and unranking the lamb set.

Aliases. When W_o is fixed by a non-trivial symmetry, some lamb masks ℓ' are not minimal under the stabiliser of W_o ; their entries are never the target of a lookup. The solver skips them during processing (Section 5) and fills them at the end by copying the value of their canonical twin, so that any index decodes to a valid entry. About 10 % of entries are aliases.

Enumeration. All lamb sets of a layer are visited in rank order by enumerating the 22-bit masks with k bits in increasing numeric order (Gosper's hack), which coincides with colex order; the rank is then just a running counter. A 2300-entry table maps raw triples to orbits, and per orbit three 256-entry tables expand a compressed 22-bit mask to a 25-bit board mask.

3 Layers

Captures remove a lamb and nothing adds one, so the position graph is stratified by k : within a layer only steps occur, and captures go from layer k to layer $k - 1$. Layers with $k \leq 3$ are wolf wins by rule. The solver processes $k = 4, 5, \dots, 15$ in order and keeps in memory only the current layer and the finished layer $k - 1$.

4 The value of a state under the rules

For side X and a state s of layer k define $t_X(c)$ for each legal successor c of s :

$$t_X(c) = \begin{cases} T_X(c) & c \in \text{Steps}(s), \\ 0 & c \in \text{Caps}(s) \text{ and } V_{k-1}(c) = X, \\ \infty & c \in \text{Caps}(s) \text{ and } V_{k-1}(c) \neq X, \end{cases}$$

and let T_X be the least solution of

$$T_X(s) = \begin{cases} 0 & \text{no legal move, mover}(s) \neq X, \\ \infty & \text{no legal move, mover}(s) = X, \\ 1 + \min_c t_X(c) & \text{mover}(s) = X, \\ 1 + \max_c t_X(c) & \text{mover}(s) \neq X. \end{cases} \quad (1)$$

$T_X(s)$ is the number of plies within which X can force, against any defence, either a terminal win or a capture into a layer- $(k - 1)$ state that is an X win. The value with a fresh no-capture counter is

$$V_k(s) = W \text{ if } T_W(s) \leq 100, \quad L \text{ if } T_L(s) \leq 100, \quad D \text{ otherwise.}$$

T_W and T_L cannot both be finite. The two rules left out of the state are handled by this definition: the counter resets at every capture, so “within 100 plies inside the layer” is exactly what the rule requires; and along a forced line T_X strictly decreases, so no position repeats, while a defender who has $T_Y > 100 - c$ for the attacker can always keep it so and reach the clock (or a repetition, also a draw). Hence X wins from s with c plies on the counter iff $T_X(s) \leq 100 - c$.

5 The retrograde pass

Equation (1) is solved by breadth-first search backwards from the exits, once per side X . Values are assigned in increasing order of T_X , which makes the first assignment to an X -to-move state its minimum and the assignment to a defender state (made when its last successor has been valued) its maximum. Algorithm 1 is the exact procedure; *distinct canonical* means that successors or predecessors that map to the same table index are counted once.

Algorithm 1 One pass of `solve.c`: T_X for layer k , side X

```

1:  $T[\cdot] \leftarrow \text{unset}$ ;  $\text{cnt}[\cdot] \leftarrow 0$ ; levels  $Q_0, Q_1 \leftarrow \emptyset$ 
2: for all non-alias states  $s$  of layer  $k$  do ▷ init, parallel over orbits
3:   if  $\text{mover}(s) = X$  then
4:     if  $s$  has a capture  $c$  with  $V_{k-1}(c) = X$  then  $T[s] \leftarrow 1$ ; push  $s$  to  $Q_1$ 
5:     end if
6:   else
7:     if  $s$  has no legal move then  $T[s] \leftarrow 0$ ; push  $s$  to  $Q_0$ 
8:     else if  $s$  has a capture  $c$  with  $V_{k-1}(c) \neq X$  then  $\text{cnt}[s] \leftarrow \text{ESCAPE}$ 
9:     else if  $\text{Steps}(s) = \emptyset$  then  $T[s] \leftarrow 1$ ; push  $s$  to  $Q_1$ 
10:    else  $\text{cnt}[s] \leftarrow |\{\text{distinct canonical } c \in \text{Steps}(s)\}|$ 
11:    end if
12:  end if
13: end for
14: for  $d = 0, 1, 2, \dots$  while  $Q_d \cup Q_{d+1} \neq \emptyset$  do
15:   for all  $s \in Q_d$  do
16:    if  $d \leq 100$  then  $V_k[s] \leftarrow X$  (error if already set by the other side)
17:    end if
18:    for all distinct canonical  $q \in \text{Preds}(s)$  do ▷  $q$  reaches  $s$  by a step
19:      if  $\text{mover}(q) = X$  and  $T[q]$  unset then  $T[q] \leftarrow d + 1$ ; push  $q$  to  $Q_{d+1}$ 
20:      else if  $\text{mover}(q) \neq X$  and  $\text{cnt}[q] \neq \text{ESCAPE}$  then  $\text{cnt}[q] \leftarrow \text{cnt}[q] - 1$ ; if  $\text{cnt}[q] = 0$ 
21:        then  $T[q] \leftarrow d + 1$ ; push  $q$  to  $Q_{d+1}$ 
22:      end if
23:    end for
24:  end for
25: after both sides: every non-alias state with  $V_k$  unset is a draw; aliases copy their canonical twin

```

Why the counters are right. A defender state q is valued when all its step successors have been valued; the counter is initialised to the number of distinct canonical successors and decremented once per processed successor c for which q is a distinct canonical predecessor. These two counts agree: if $q \rightarrow c_1$ is a step and g a symmetry with $g(c_1)$ canonical, then $g(q) \rightarrow g(c_1)$ is a step, so “ c is a distinct canonical successor of q ” and “ q is a distinct canonical predecessor of c ” are equivalent. Aliases must be excluded from processing; otherwise a state and its mirror image would both decrement the counters of their common predecessors.

Predecessors. Within a layer the only backward move is undoing a step: for each piece of the side that just moved, and each empty neighbour of it, move the piece back. Captures are never undone inside a layer; they are the exits, valued directly from layer $k - 1$ during initialisation.

Cost. Initialisation generates the moves of every state once (parallel over wolf orbits, disjoint index ranges). Propagation touches each valued state once and each in-layer edge once. Memory per layer: the value table (2 bytes per entry), T (2 bytes), counters (1 byte), the queues, and the previous layer’s values: about 3 GB for the largest layer ($k = 11$, 450 million entries). The whole solve took about 12 minutes on 12 threads.

6 Plies to the end of the game

`dist.c` reads the value tables and, for every won state with winner X , computes

$$D(s) = \begin{cases} 0 & \text{no legal move,} \\ 1 + \min_c d(c) & \text{mover}(s) = X, \\ 1 + \max_c d(c) & \text{mover}(s) \neq X, \end{cases} \quad d(c) = \begin{cases} D(c) & \text{step,} \\ D_{k-1}(c) & \text{capture,} \\ 0 & \text{capture leaving three lambs,} \end{cases}$$

where captures are only followed into states that layer $k - 1$ labels as X wins. Because capture successors carry arbitrary distances, plain level-by-level BFS does not suffice; the pass uses a *bucket queue* indexed by distance. An X -to-move state is given a provisional distance from its best capture at initialisation and is lowered whenever a step successor with a smaller distance is processed, stale queue entries being skipped; a defender state’s distance is computed as $1 + \max$ over all successors when its counter reaches zero (all step successors valued by then, capture successors known). Processing buckets in increasing order guarantees minimality for X -to-move states and exactness for defender states.

The 100-turn rule is not modelled inside D . Instead a second level-BFS computes $S(s)$, the capture-free stretch that a D -optimal strategy needs, where the winner chooses among D -optimal moves the one minimising S and only lines whose captures land in positions that are themselves exact are counted ($S \leq 100$ in the layer below). If $S(s) \leq 100 - c$ the D -optimal strategy is legal under the clock and $D(s)$ is exact; otherwise $D(s)$ is a proven lower bound and the tools display it with $\geq$. This affects 0.38 % of won states.

7 Packing for the website

`pack.js` reads each value table and writes 2 bits per entry ($1 = W$, $2 = L$, $3 = D$) in index order, entry e at bit offset $2e$, into files of at most 64 MB (`layer_k.p0.bin`, `layer_k.p1.bin`); `index.json` records sizes and part boundaries. A lookup in the browser recomputes the index of Section 2 in JavaScript, fetches the 4 KB block containing byte $\lfloor e/4 \rfloor$ with an HTTP range request, and extracts the 2-bit field. No server is involved. All layers together are 651 MB.

8 Verification

The solver is not trusted. `verify.js` re-implements the indexing independently, generates moves with the bitboard engine (which is itself checked move-for-move against the plain rules engine on 300 000 positions), and tests at every non-alias state a local condition on the labels. Call a child c of s a “ Z -win within n ” if it is reached by a capture into a layer- $(k - 1)$ Z win, or by a step to a state labelled Z with $T \leq n$.

label of s	required property
no legal move	label = opponent, $T = 0$
mover M wins, T	some child is an M -win within $T - 1$, none within $T - 2$, $1 \leq T \leq 100$
opponent Y wins, T	every child is a Y -win within $T - 1$, some not within $T - 2$, $1 \leq T \leq 100$
draw	no child is an M -win within 99, some child is not a Y -win within 99

By induction on T these conditions force the labels to equal the true values, given a correct layer $k - 1$ and the base rule for three lambs. Aliases are checked to equal their twins, and the indexing is self-tested on random positions (all eight symmetric images map to one index, and the index decodes to an equivalent state). `verify_dist.js` checks D and S by the analogous local conditions. A separate spot check compares random entries with a brute-force search over the plain rules. All checks reported zero errors.

9 File formats

file	entry	meaning
<code>solver/tables/layer_k.bin</code>	uint16	bits 15–14: value (1 W, 2 L, 3 D); bits 13–0: T of the winner
<code>solver/tables/dist_k.bin</code>	uint16	D (0xFFFF for draws)
<code>solver/tables/seg_k.bin</code>	uint16	S (0xFFFF for draws, or when no exact D -optimal line exists)
<code>tables/layer_k.pN.bin</code>	2 bits	value only, for the website

Entry order in every file is the index of Section 2; all integers are little-endian.